

R for Data Science exercises

Atividade semanal - Statistical Inference I

Bruna Marques Moreno

1 Introduction

Read Chapter 10 of R for Data Science and solve all exercises. Build a report using Quarto or R Markdown and render it in pdf or html.

2 Exercises 10.3.3

2.1 Explore the distribution of each of the x, y, and z variables in diamonds. What do you learn? Think about a diamond and how you might decide which dimension is the length, width, and depth.

```
library(tidyverse)
data("diamonds")
head(diamonds)
```

```
# A tibble: 6 x 10
  carat cut      color clarity depth table price      x      y      z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal    E     SI2     61.5   55   326   3.95   3.98   2.43
2  0.21 Premium E     SI1     59.8   61   326   3.89   3.84   2.31
3  0.23 Good    E     VS1     56.9   65   327   4.05   4.07   2.31
4  0.29 Premium I     VS2     62.4   58   334   4.2    4.23   2.63
5  0.31 Good    J     SI2     63.3   58   335   4.34   4.35   2.75
6  0.24 Very Good J     VVS2     62.8   57   336   3.94   3.96   2.48
```

```
str(diamonds)
```

```
tibble [53,940 x 10] (S3: tbl_df/tbl/data.frame)
 $ carat  : num [1:53940] 0.23 0.21 0.23 0.29 0.31 0.24 0.24 0.26 0.22 0.23 ...
 $ cut    : Ord.factor w/ 5 levels "Fair"<"Good"<...: 5 4 2 4 2 3 3 3 1 3 ...
 $ color  : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<...: 2 2 2 6 7 7 6 5 2 5 ...
 $ clarity: Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<...: 2 3 5 4 2 6 7 3 4 5 ...
 $ depth  : num [1:53940] 61.5 59.8 56.9 62.4 63.3 62.8 62.3 61.9 65.1 59.4 ...
 $ table  : num [1:53940] 55 61 65 58 58 57 57 55 61 61 ...
 $ price  : int [1:53940] 326 326 327 334 335 336 336 337 337 338 ...
```

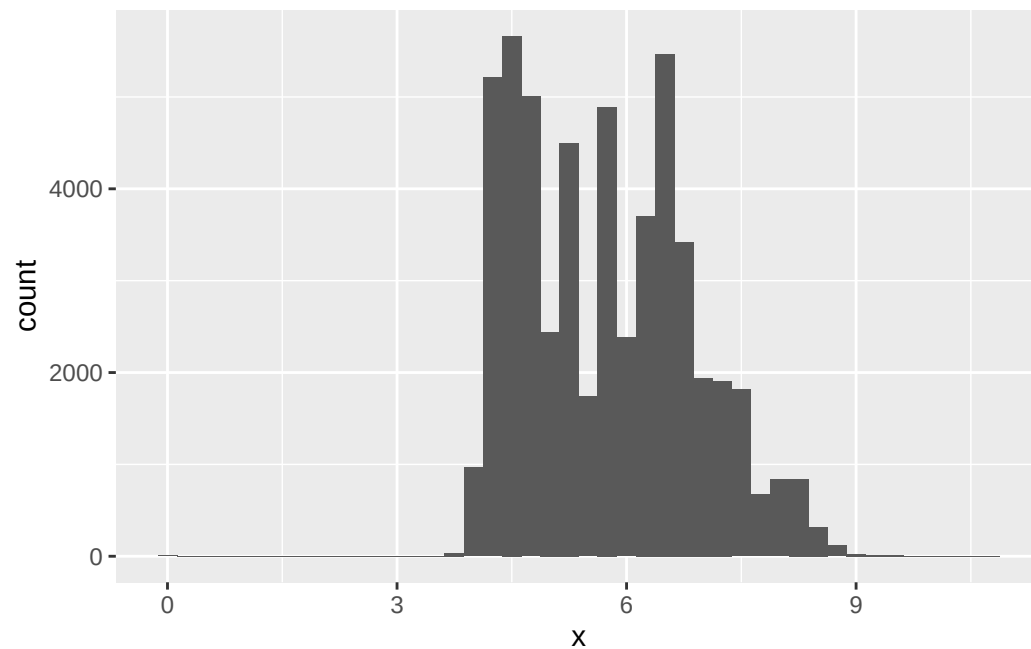
```
$ x      : num [1:53940] 3.95 3.89 4.05 4.2 4.34 3.94 3.95 4.07 3.87 4 ...
$ y      : num [1:53940] 3.98 3.84 4.07 4.23 4.35 3.96 3.98 4.11 3.78 4.05 ...
$ z      : num [1:53940] 2.43 2.31 2.31 2.63 2.75 2.48 2.47 2.53 2.49 2.39 ...
```

2.1.1 Analyzing X

```
summary(diamonds$x)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 0.000   4.710   5.700   5.731   6.540  10.740
```

```
ggplot(diamonds, aes(x = x)) +
  geom_histogram(binwidth = 0.25)
```



```
diamonds |>
  filter (x > 10)
```

```
# A tibble: 5 x 10
```

	carat	cut	color	clarity	depth	table	price	x	y	z
	<dbl>	<ord>	<ord>	<ord>	<dbl>	<dbl>	<int>	<dbl>	<dbl>	<dbl>
1	4.01	Premium	I	I1	61	61	15223	10.1	10.1	6.17
2	4.01	Premium	J	I1	62.5	62	15223	10.0	9.94	6.24
3	4	Very Good	I	I1	63.3	58	15984	10.0	9.94	6.31
4	5.01	Fair	J	I1	65.5	59	18018	10.7	10.5	6.98
5	4.5	Fair	J	I1	65.8	58	18531	10.2	10.2	6.72

```
diamonds |>
  filter(x < 3 | x > 10) |>
  select(price, carat, x, y, z) |>
  arrange(y)
```

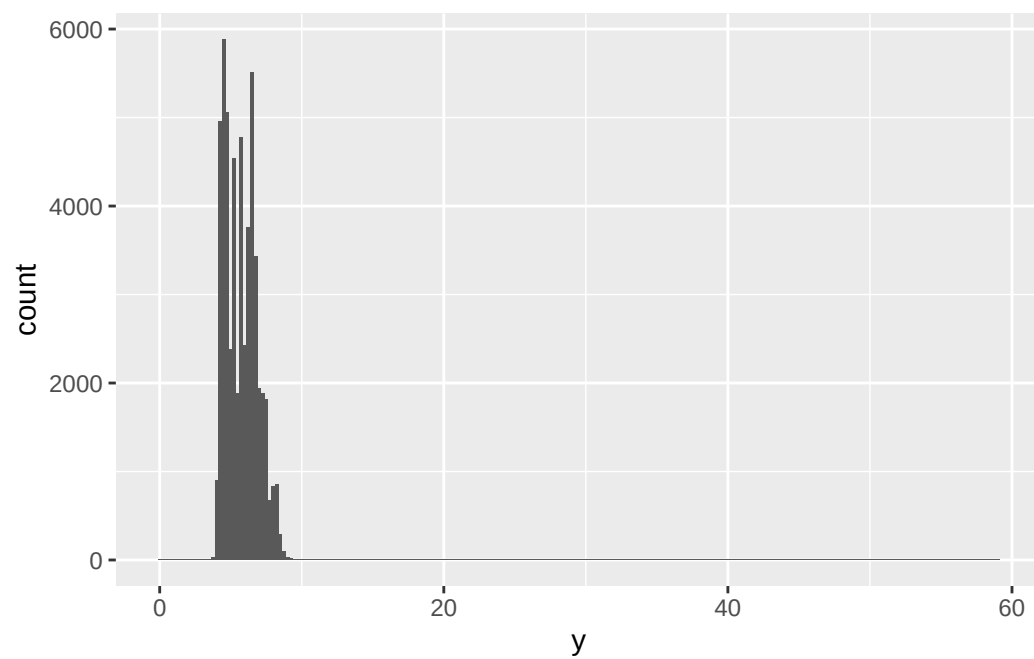
```
# A tibble: 13 x 5
  price carat      x      y      z
  <int> <dbl> <dbl> <dbl> <dbl>
1  5139     1     0     0     0
2  6381   1.14     0     0     0
3 12800   1.56     0     0     0
4 15686   1.2     0     0     0
5 18034   2.25     0     0     0
6  2130   0.71     0     0     0
7  2130   0.71     0     0     0
8  4954   1.07     0    6.62     0
9 15223   4.01   10.0   9.94   6.24
10 15984    4    10.0   9.94   6.31
11 15223   4.01   10.1  10.1   6.17
12 18531   4.5    10.2  10.2   6.72
13 18018   5.01   10.7  10.5   6.98
```

2.1.2 Analyzing y

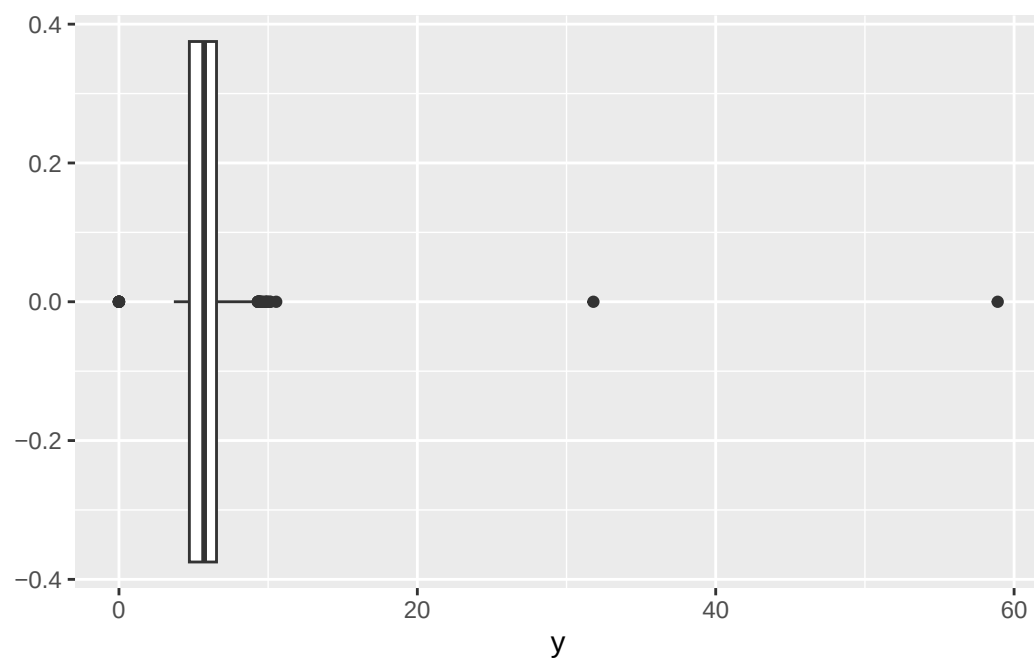
```
summary(diamonds$y)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.000   4.720   5.710   5.735   6.540   58.900
```

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.25)
```



```
ggplot(diamonds, aes(x = y)) +  
  geom_boxplot()
```



```
max(diamonds$y)
```

```
[1] 58.9
```

```
diamonds |>  
  filter (y > 10) |>  
  select(cut)
```

```
# A tibble: 5 x 1
  cut
<ord>
1 Premium
2 Premium
3 Fair
4 Fair
5 Ideal
```

```
diamonds |>
  filter(y < 3 | y > 11) |>
  select(price, carat, x, y, z, cut) |>
  arrange(y)
```

```
# A tibble: 9 x 6
  price carat      x      y      z cut
<int> <dbl> <dbl> <dbl> <dbl> <ord>
1  5139  1      0      0      0 Very Good
2  6381 1.14    0      0      0 Fair
3 12800 1.56    0      0      0 Ideal
4 15686 1.2     0      0      0 Premium
5 18034 2.25    0      0      0 Premium
6  2130 0.71    0      0      0 Good
7  2130 0.71    0      0      0 Good
8  2075 0.51   5.15   31.8   5.12 Ideal
9 12210 2       8.09   58.9   8.06 Premium
```

The y measurement is different. Analyzing what seems to be outliers (values under 3 and above 11), the values look like they are miswritten values. There is no reason to have a large height and a lower value of carat.

I believe that x is the width, y is height and z is depth because the values of x and y follow a same pattern of size. I think that what is significantly different is the depth (the radius, depending on the cut) of the diamond.

2.2 Explore the distribution of price. Do you discover anything unusual or surprising? (Hint: Carefully think about the binwidth and make sure you try a wide range of values.)

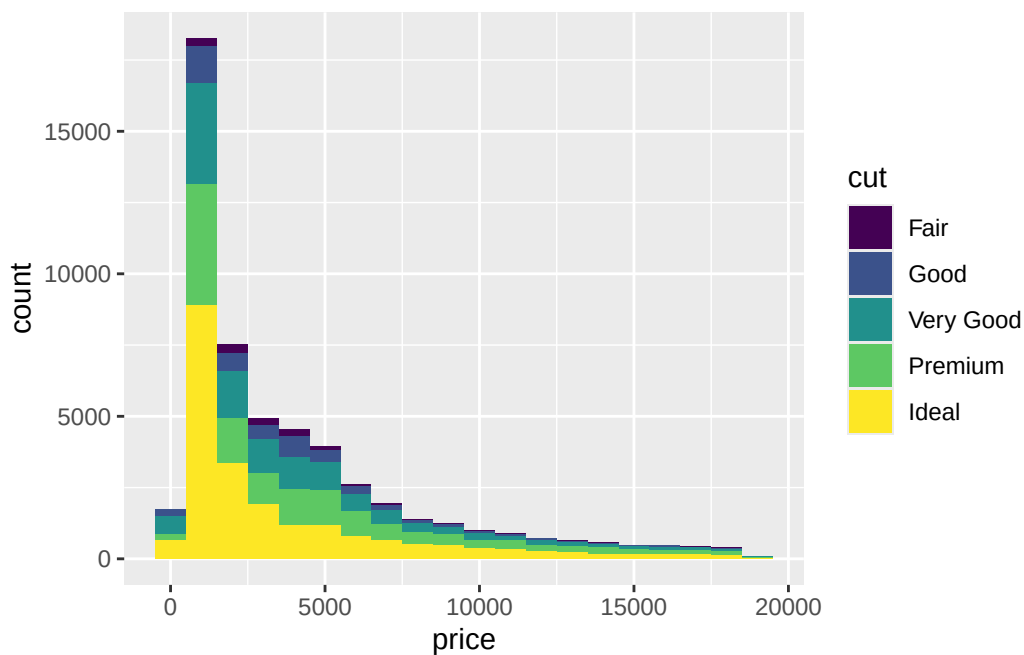
```
summary(diamonds$price)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
326	950	2401	3933	5324	18823

```
diamonds |>
  select(carat, cut, price) |>
  arrange(cut)
```

```
# A tibble: 53,940 x 3
  carat cut    price
  <dbl> <ord> <int>
1  0.22 Fair    337
2  0.86 Fair   2757
3  0.96 Fair   2759
4  0.7  Fair   2762
5  0.7  Fair   2762
6  0.91 Fair   2763
7  0.91 Fair   2763
8  0.98 Fair   2777
9  0.84 Fair   2782
10 1.01 Fair   2788
# i 53,930 more rows
```

```
ggplot(data = diamonds, aes(x= price, fill = cut)) +
  geom_histogram(binwidth = 1000)
```



I discovered that the higher the price more common is the cut Ideal. Analyzing the distribution of price, the largest group is diamonds which is \$ 100-200. I always thought diamonds were more expensive. Also, not so noble cuts can also be expensive.

2.3 How many diamonds are 0.99 carat? How many are 1 carat? What do you think is the cause of the difference?

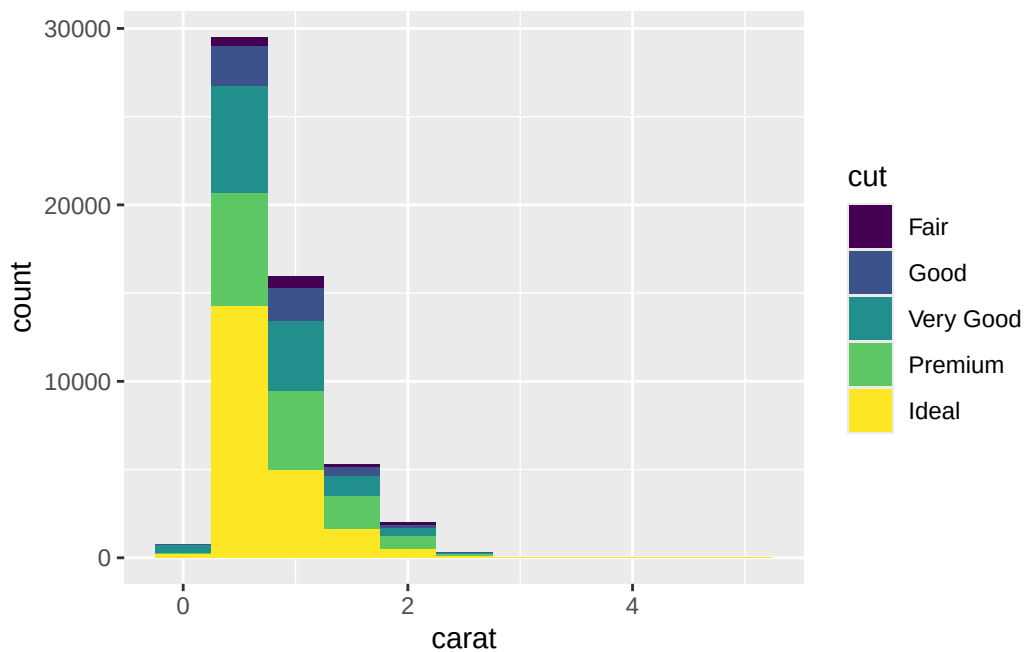
```
diamonds |>
  filter (carat == 0.99) |>
  count()
```

```
# A tibble: 1 x 1
      n
  <int>
1    23
```

```
diamonds |>
  filter (carat == 1) |>
  count()
```

```
# A tibble: 1 x 1
      n
  <int>
1  1558
```

```
ggplot(data = diamonds, aes(x= carat, fill = cut)) +
  geom_histogram(binwidth = 0.5)
```

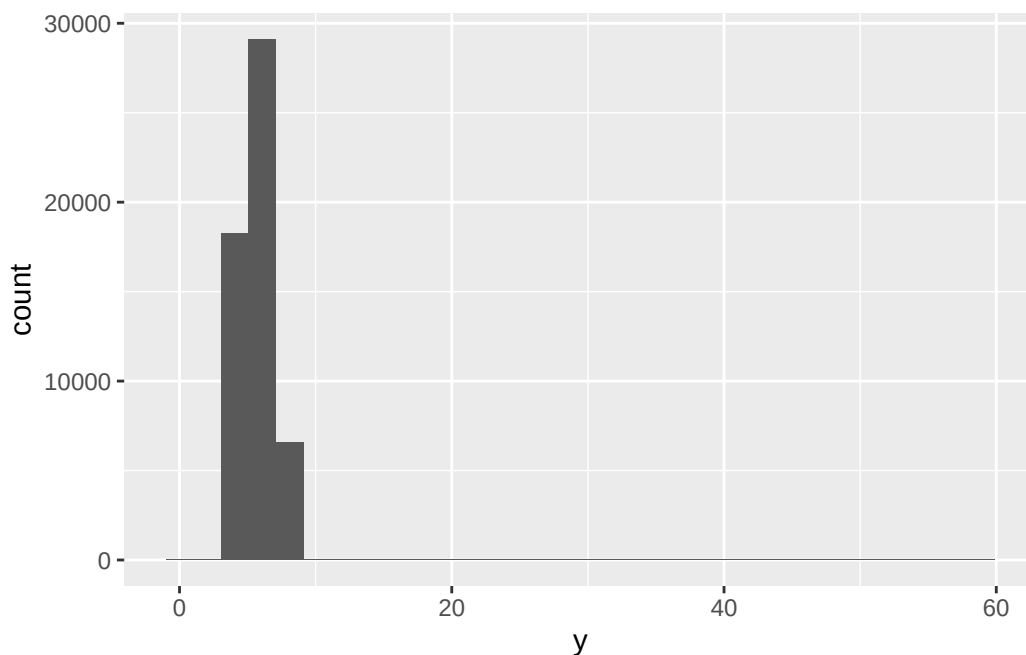


There are 23 diamonds 0.99 carat, and 1,558 1 carat. I believe that the reason to the difference may be the balance used to measure. It is a very small difference, that depends of the equipment used and who measured the diamonds.

2.4 Compare and contrast `coord_cartesian()` vs. `xlim()` or `ylim()` when zooming in on a histogram. What happens if you leave `binwidth` unset? What happens if you try and zoom so only half a bar shows?

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram()
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



Both functions are used to zoom in or out of the graph to focus on the most informative part. However, the main difference between them is that `coord_cartesian()` actually zooms in on the graph without deleting data. In contrast, the `xlim()` and `ylim()` arguments exclude data from the graph, making it impossible to evaluate that data afterward.

When the binwidth is not set, `ggplot2` chooses a default value. Zooming in is a strategy to determine if certain values are so low that they are not visible. If a bar is only half-shown, depending on your objective, you should rethink the zoom level or consider if you are losing critical information by leaving it that way.

3 Exercises 10.4.1

3.1 What happens to missing values in a histogram? What happens to missing values in a bar chart? Why is there a difference in how missing values are handled in histograms and bar charts?

```
diamonds |>
  count (is.na(carat))
```

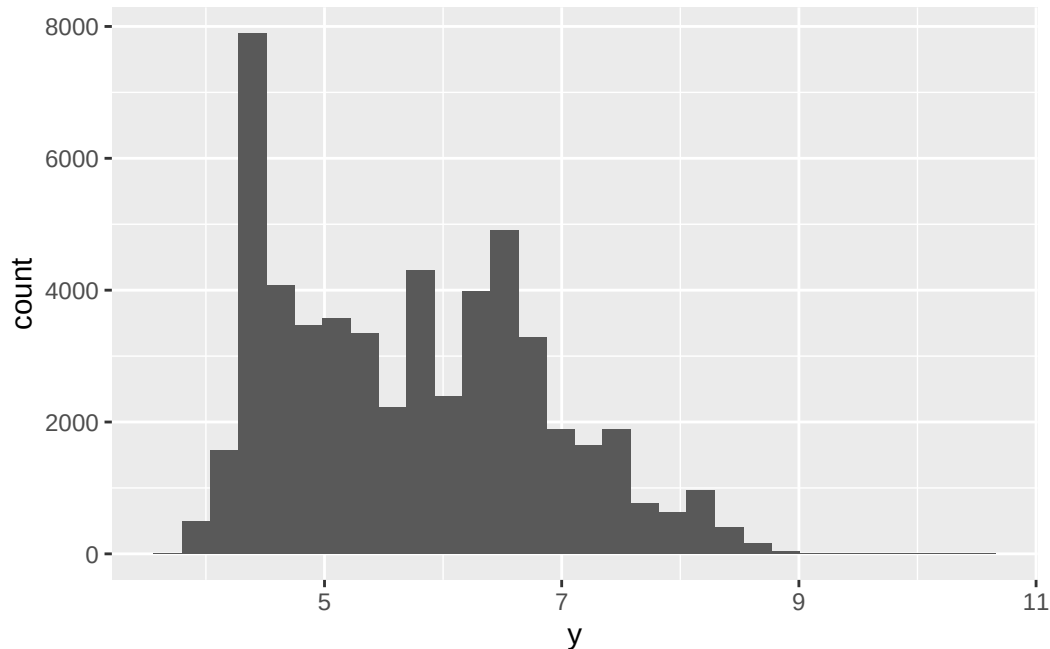
```
# A tibble: 1 x 2
  `is.na(carat)`      n
  <lgl>           <int>
1 FALSE         53940
```

```
diamonds2 <- diamonds |>
  mutate(y = if_else(y < 3 | y > 20, NA, y))

ggplot(diamonds2, aes(x = y)) +
  geom_histogram()
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.

Warning: Removed 9 rows containing non-finite outside the scale range (``stat_bin()``).



When missing values are treated as zeros in the data frame, the histogram will show a higher count for zero, or within the bin where zero is located. However, when there are NAs in the data frame, ggplot2 will exclude those values. As seen in the image above, there are no zeros. In the case of bar graphs, the height of a given category will be lower than it should be, since those missing values are not accounted for.

3.2 What does `na.rm = TRUE` do in `mean()` and `sum()`?

In both cases, NA values are removed. As a consequence, the mean will only reflect samples that are neither zero nor NA. This also impacts the result of `sum()`; since the argument removes NAs and they are not counted, the total sum will be lower than expected.

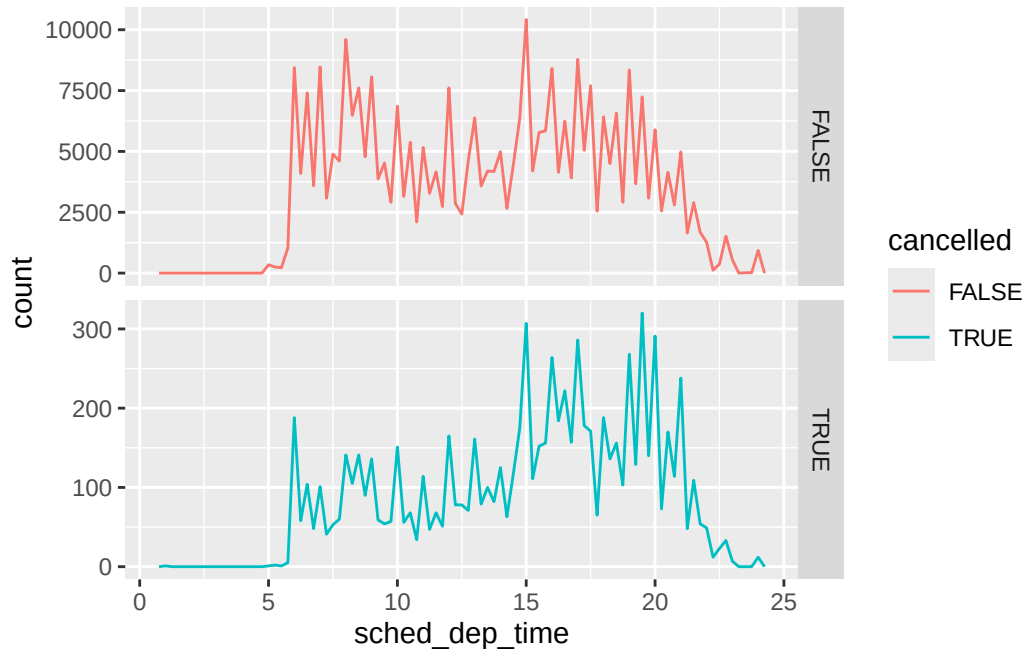
3.3 Recreate the frequency plot of `scheduled_dep_time` colored by whether the flight was cancelled or not. Also facet by the cancelled variable. Experiment with different values of the scales variable in the faceting function to mitigate the effect of more non-cancelled flights than cancelled flights.

```
#install.packages("nycflights13")
library("nycflights13")
nycflights13::flights |>
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %/% 100,
```

```

    sched_dep_time = sched_hour + (sched_min / 60)
  ) |>
  ggplot(aes(x = sched_dep_time)) +
  geom_freqpoly(aes(color = cancelled), binwidth = 1/4) +
  facet_grid(cancelled ~ ., scales = "free_y")

```



It is interesting because they present very close pattern even though the scale is very different.

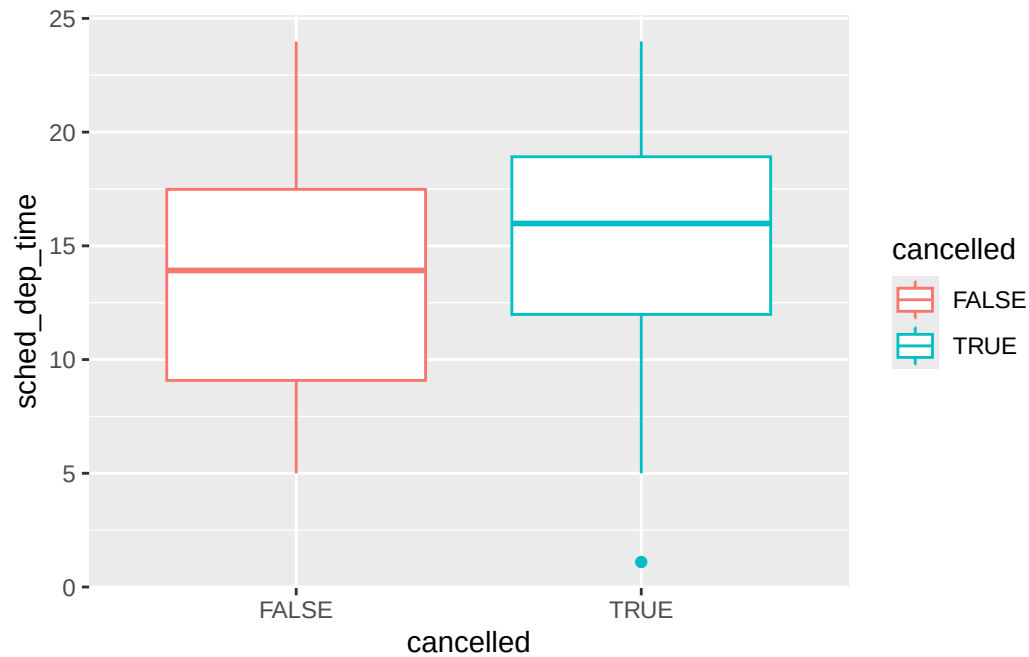
4 Exercises 10.5.1.1

4.1 Use what you've learned to improve the visualization of the departure times of cancelled vs. non-cancelled flights.

```

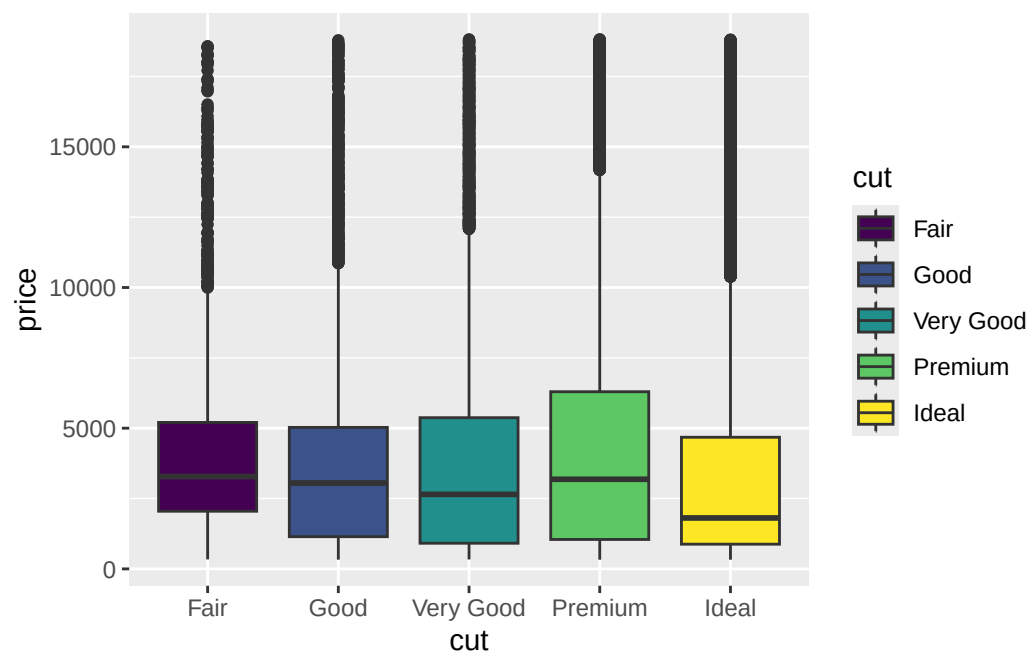
library("nycflights13")
nycflights13::flights |>
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + (sched_min / 60)
  ) |>
  ggplot(aes(x = cancelled, y = sched_dep_time)) +
  geom_boxplot(aes(color = cancelled))

```

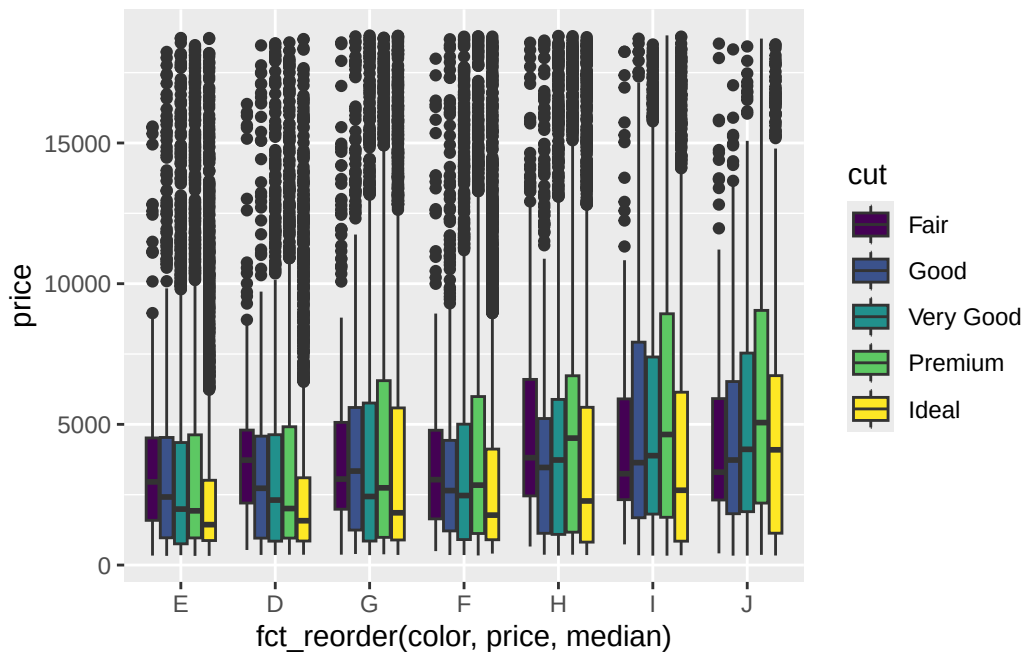


4.2 Based on EDA, what variable in the diamonds dataset appears to be most important for predicting the price of a diamond? How is that variable correlated with cut? Why does the combination of those two relationships lead to lower quality diamonds being more expensive?

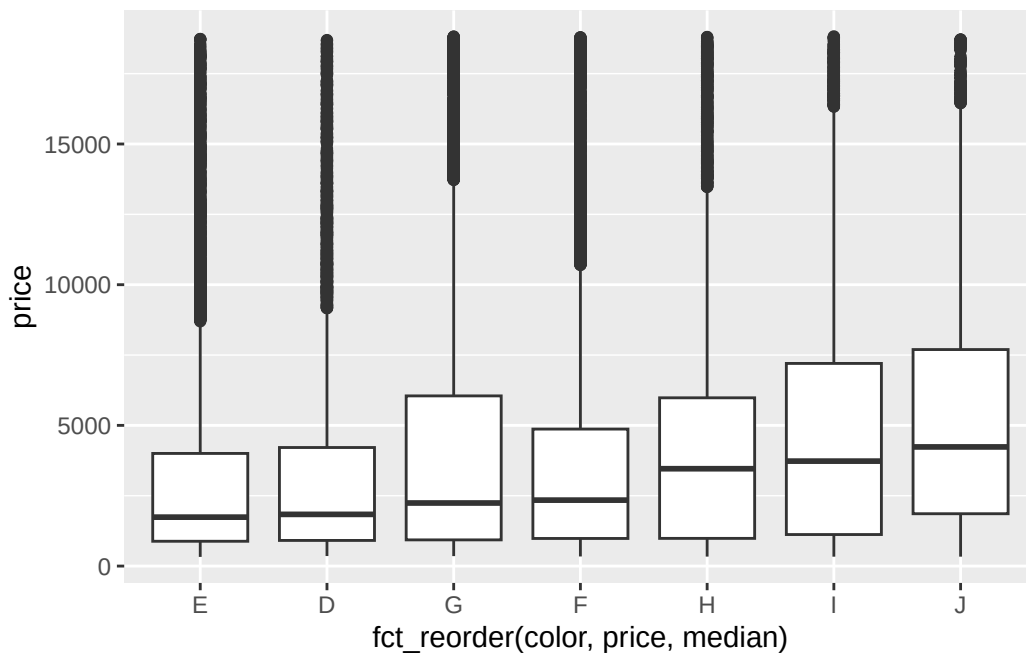
```
library("ggplot2")
ggplot(diamonds, aes(x = cut, y = price, fill = cut))+
  geom_boxplot()
```



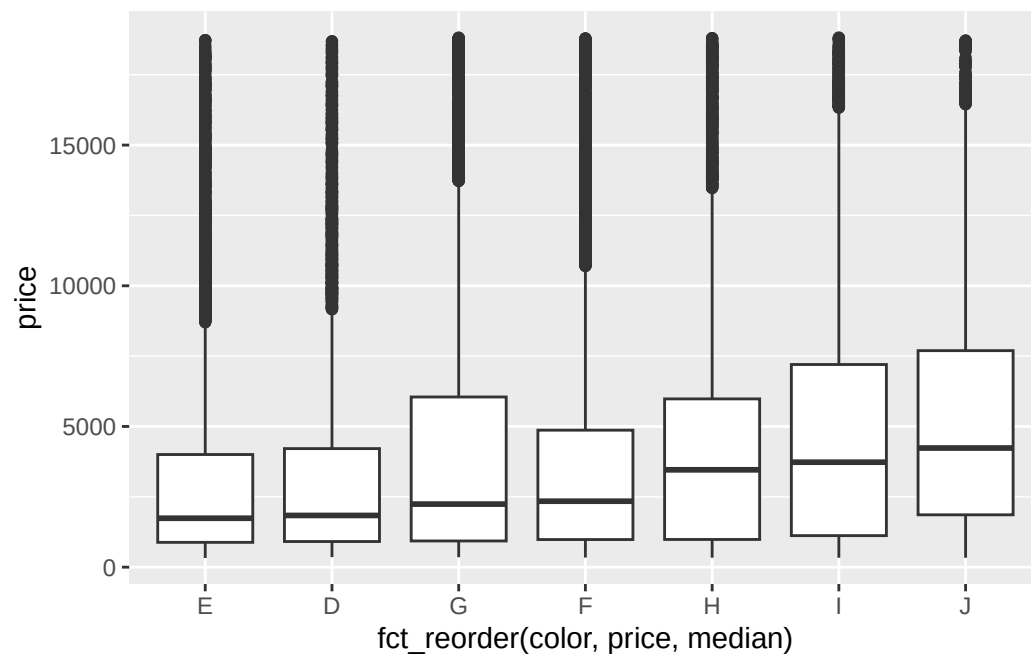
```
ggplot(diamonds, aes(x = fct_reorder(color, price, median), y = price, fill = cut)) +  
  geom_boxplot()
```



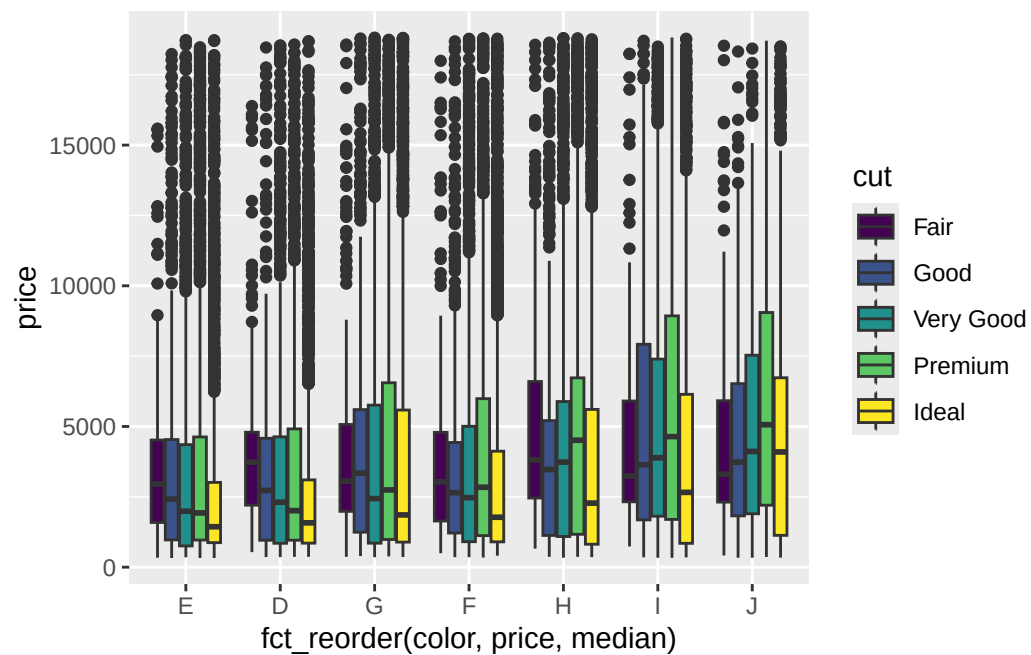
```
ggplot(diamonds, aes(x = fct_reorder(color, price, median), y = price)) +  
  geom_boxplot()
```



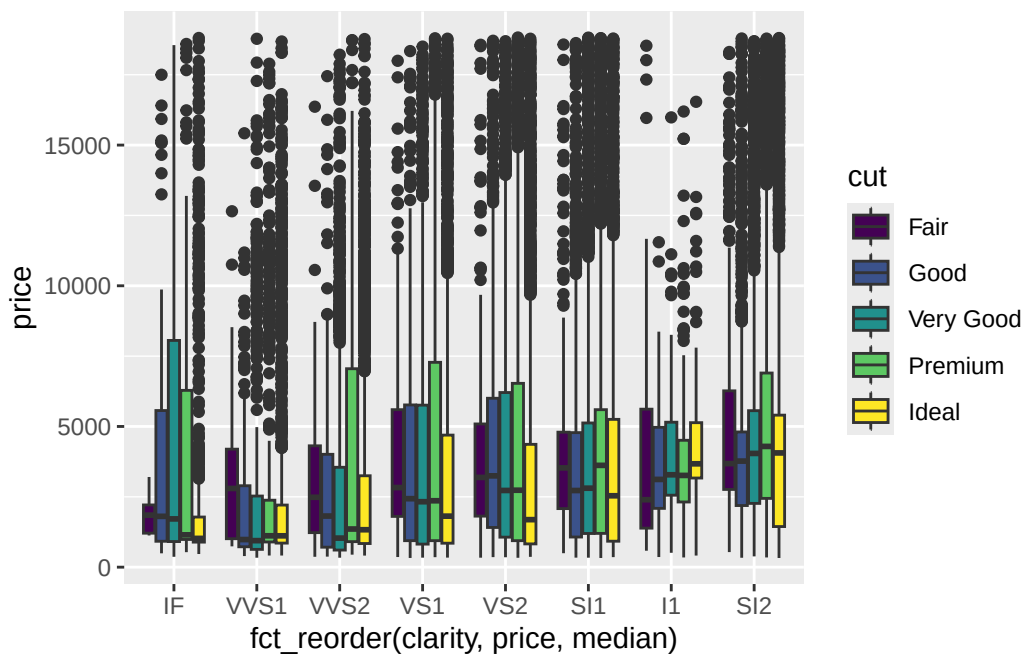
```
ggplot(diamonds, aes(x = fct_reorder(color, price, median), y = price)) +  
  geom_boxplot()
```



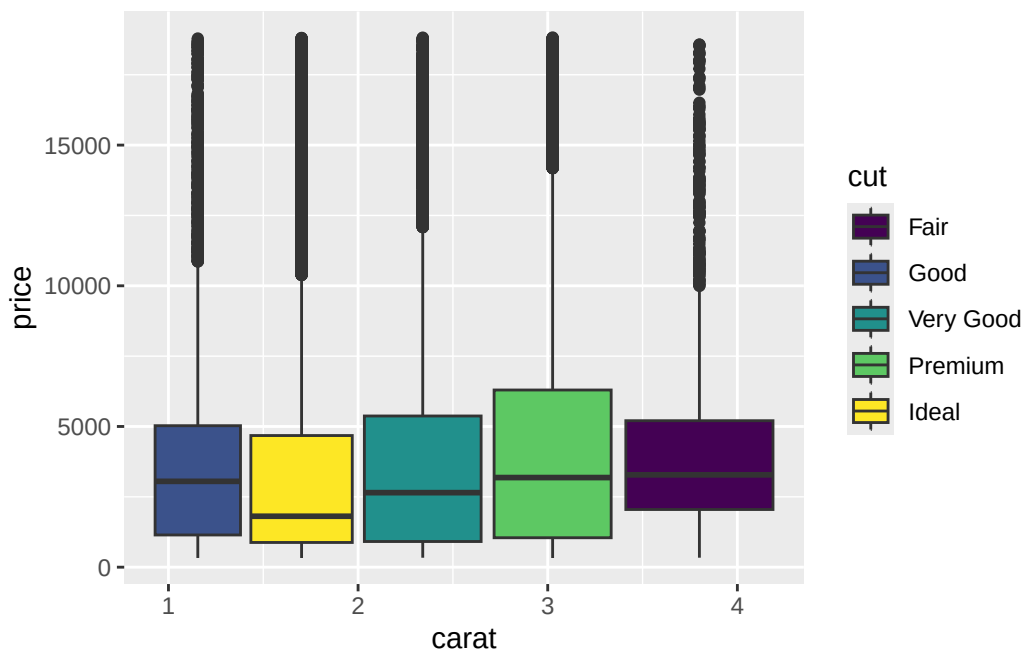
```
ggplot(diamonds, aes(x = fct_reorder(color, price, median), y = price, fill = cut)) +
  geom_boxplot()
```



```
ggplot(diamonds, aes(x = fct_reorder(clarity, price, median), y = price, fill = cut)) +
  geom_boxplot()
```

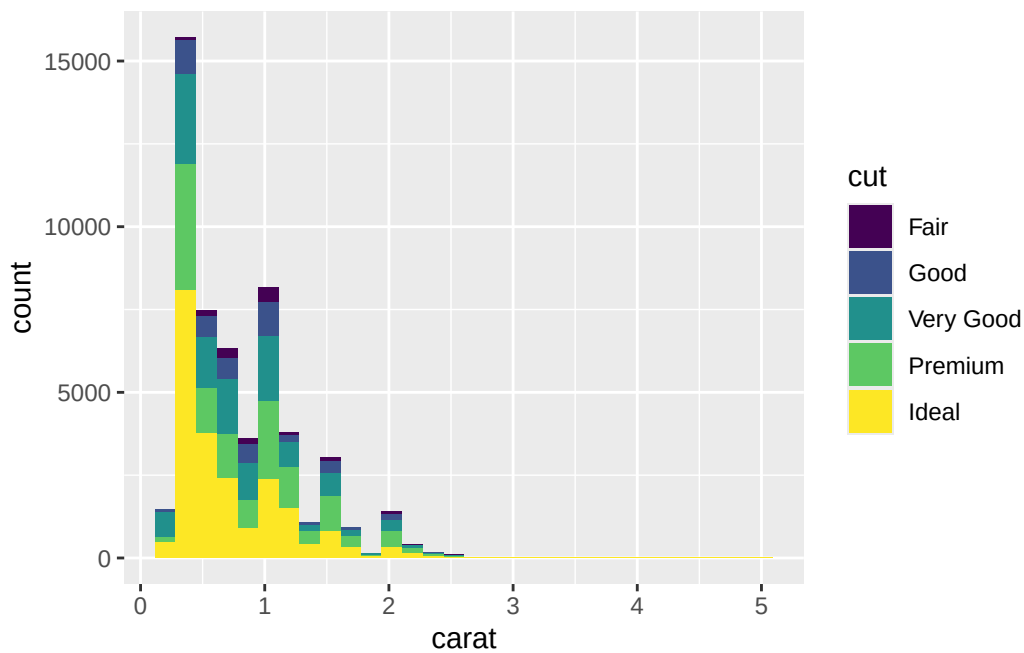


```
ggplot(diamonds, aes(x = carat, y = price, fill = cut))+
  geom_boxplot()
```

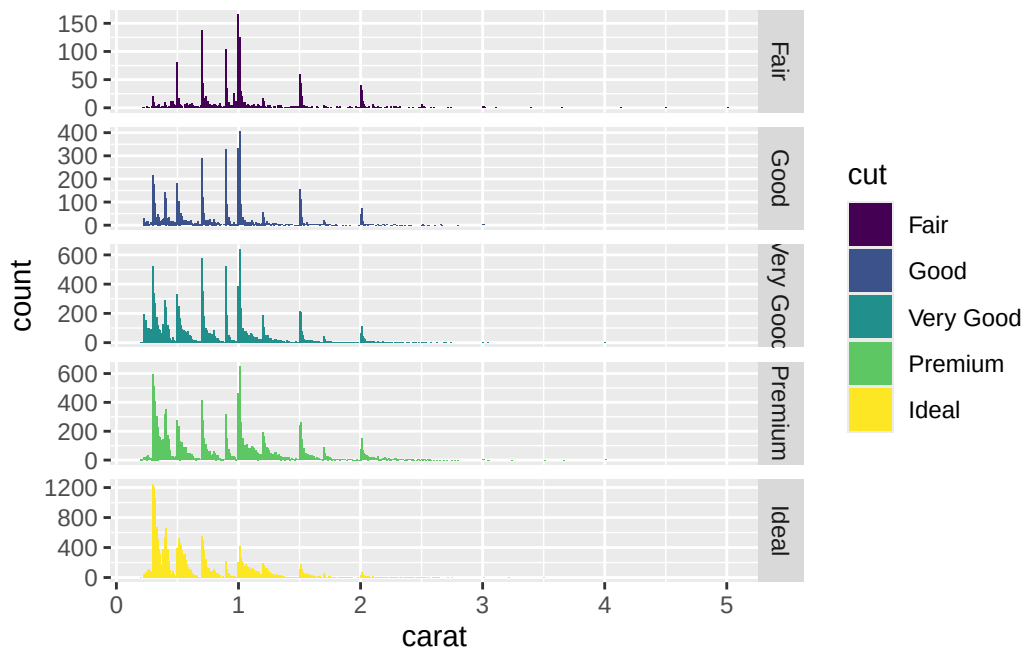


```
ggplot(diamonds, aes(x = carat, fill = cut))+
  geom_histogram()
```

``stat_bin()`` using ``bins = 30``. Pick better value ``binwidth``.



```
ggplot(diamonds, aes(x = carat, fill = cut))+
  geom_bar()+
  facet_grid(cut ~ ., scales = "free_y")
```

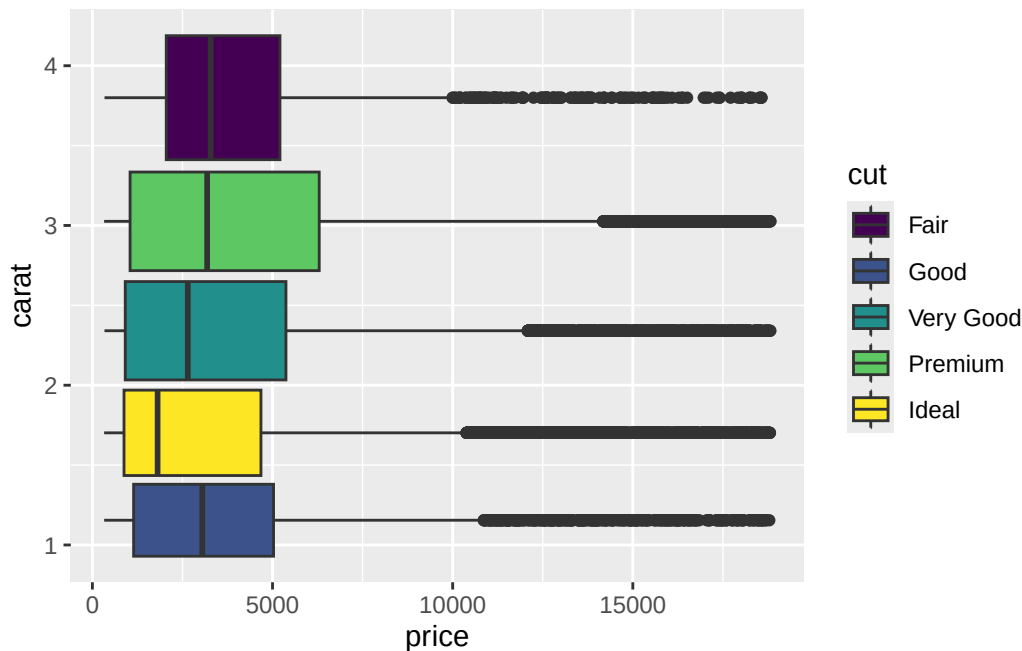


Evaluating the relationship between price and cut, it appears that lower quality of cut is more expensive. But also, the color of the diamond is also correlated. The color J is more expensive than color E. When the analysis considers color and cut, medium-high quality in color present higher prices.

Also, the better quality is the diamond, smaller its carat is because it was polished. The *fair* diamonds are heavier, then it costs more.

4.3 Instead of exchanging the x and y variables, add `coord_flip()` as a new layer to the vertical boxplot to create a horizontal one. How does this compare to exchanging the variables?

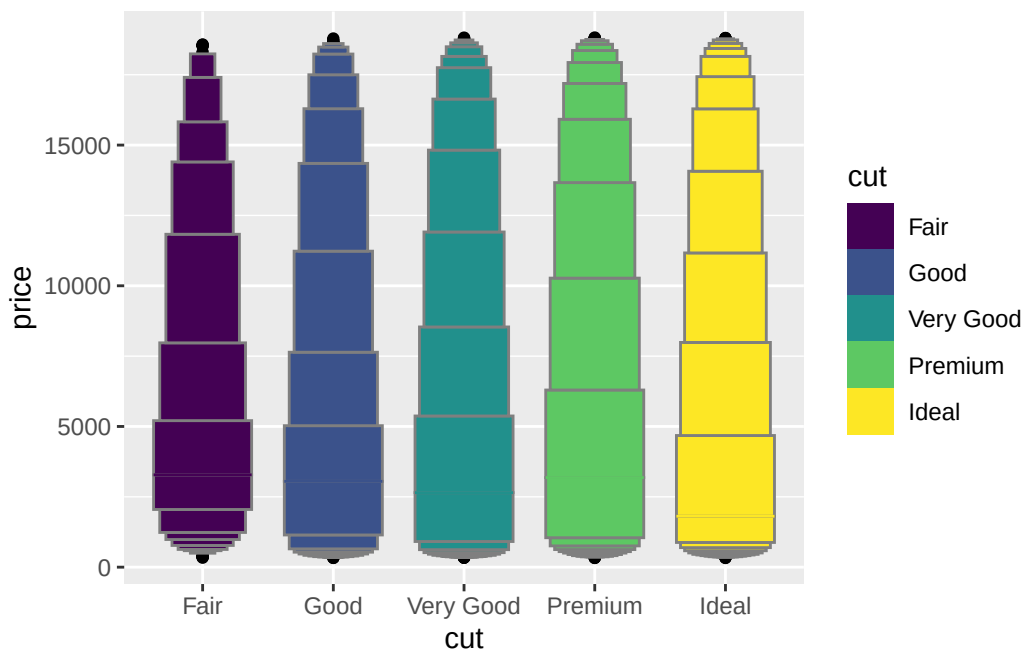
```
ggplot(diamonds, aes(x = carat, y = price, fill = cut))+  
  geom_boxplot()+  
  coord_flip()
```



Adding `coord_flip()` produces the same effect as swapping the x and y variables. However, the visual result differs. The `coord_flip()` function is particularly helpful when dealing with long variable names, as it improves readability.

4.4 One problem with boxplots is that they were developed in an era of much smaller datasets and tend to display a prohibitively large number of “outlying values”. One approach to remedy this problem is the letter value plot. Install the `lvplot` package, and try using `geom_lv()` to display the distribution of price vs. cut. What do you learn? How do you interpret the plots?

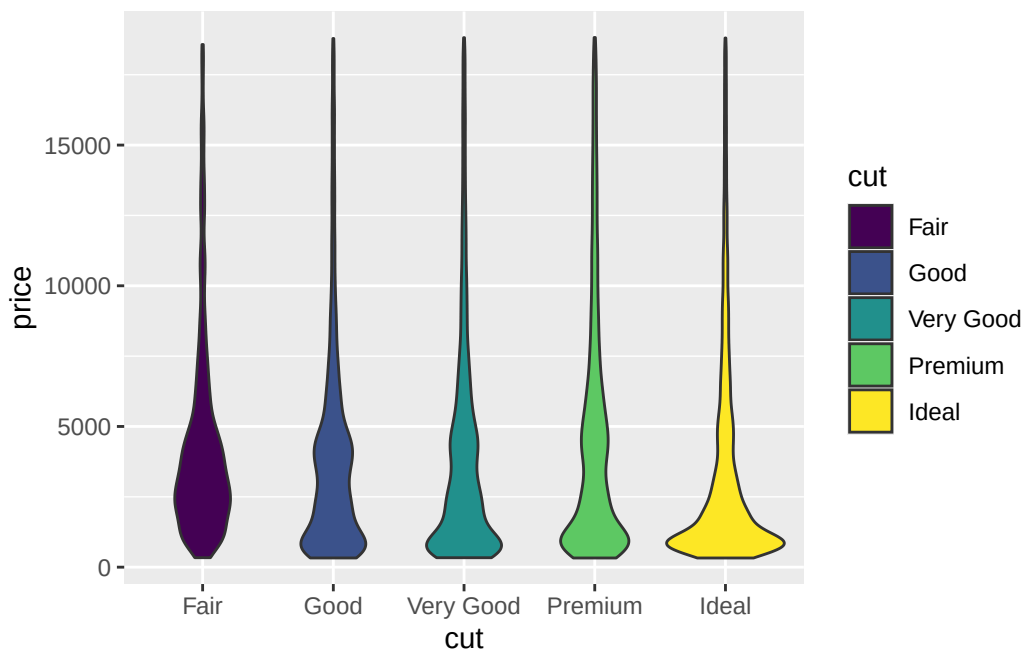
```
#install.packages("lvplot")  
library(lvplot)  
?geom_lv  
ggplot(data = diamonds, aes(x = cut, y = price, fill = cut))+  
  geom_lv()
```



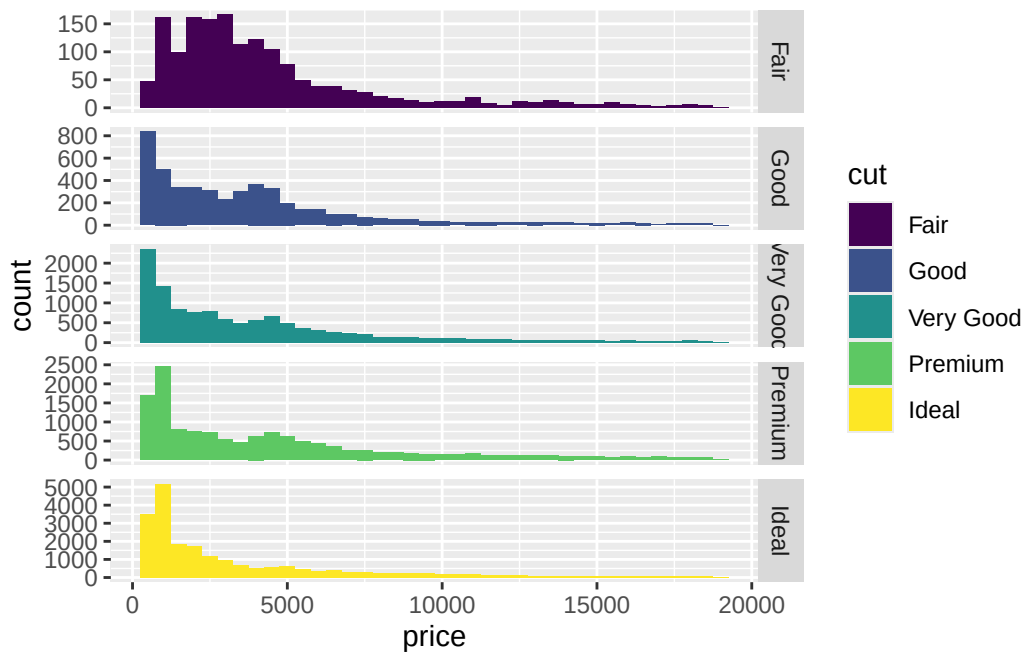
The output graph is definitely different. I interpret it such that the larger the section, the higher the number of observations with those values. It differs from a traditional boxplot because the quantiles are more flexible. According to the function's help section, I now understand the limitations of the traditional boxplot proposed by Tukey. While smaller datasets can be easily divided into quartiles, larger ones tend to have more extreme values, which can inflate the number of points labeled as outliers even when they aren't 'true' outliers.

4.5 Create a visualization of diamond prices vs. a categorical variable from the diamonds dataset using `geom_violin()`, then a faceted `geom_histogram()`, then a colored `geom_freqpoly()`, and then a colored `geom_density()`. Compare and contrast the four plots. What are the pros and cons of each method of visualizing the distribution of a numerical variable based on the levels of a categorical variable?

```
ggplot(data = diamonds, aes(x = cut, y = price, fill = cut))+
  geom_violin()
```

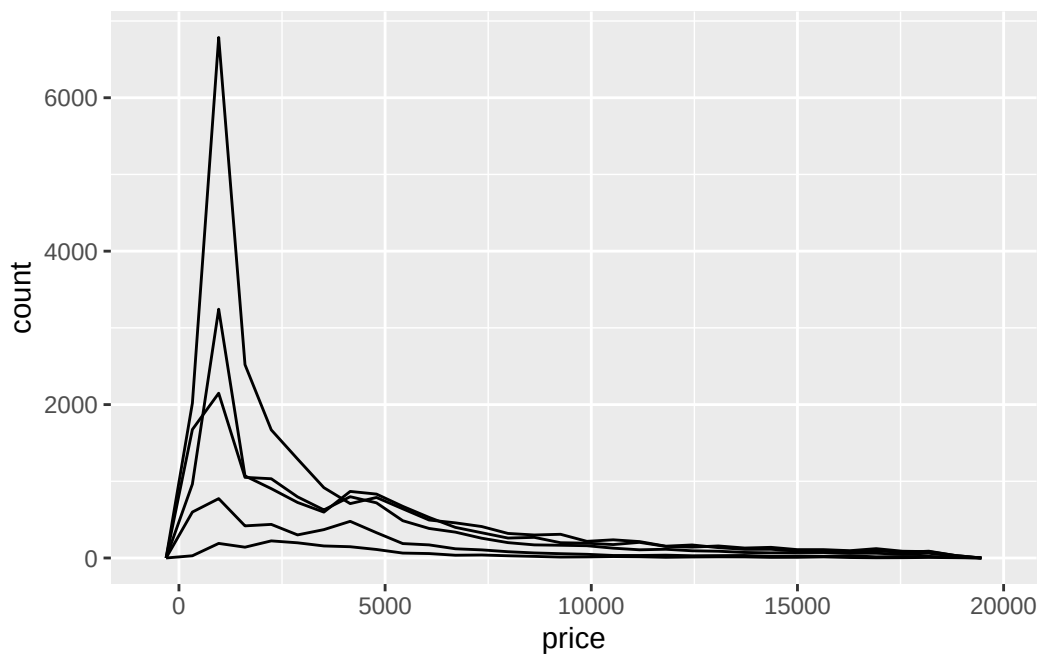


```
ggplot(data = diamonds, aes(x = price, fill = cut))+
  geom_histogram(binwidth = 500)+
  facet_grid(cut ~ ., scale = "free_y")
```

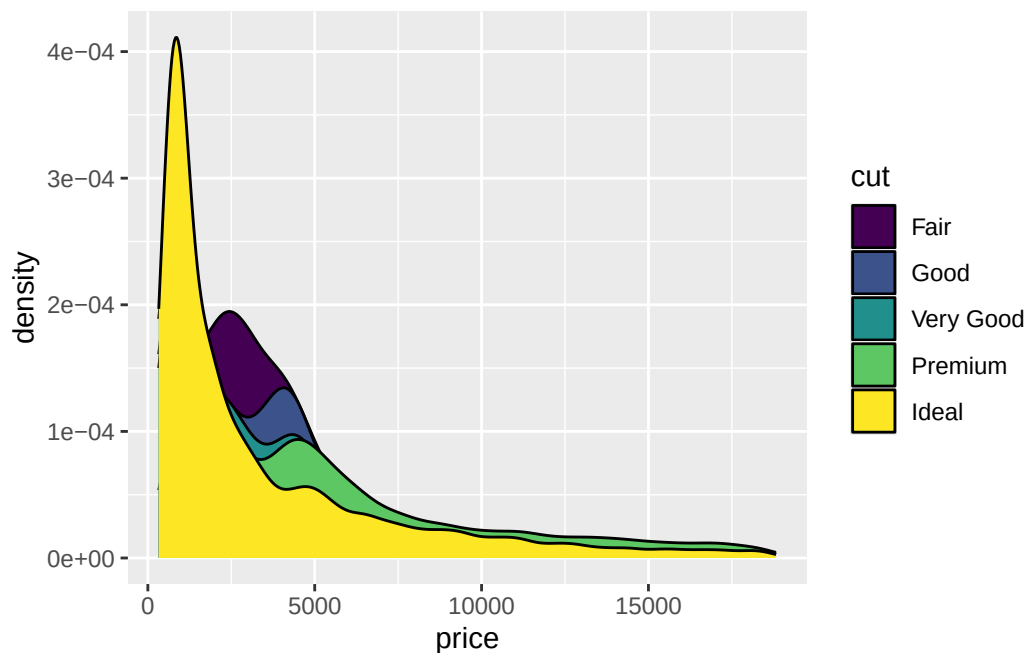


```
ggplot(data = diamonds, aes(x = price, fill = cut))+
  geom_freqpoly()
```

`stat\_bin()` using `bins = 30`. Pick better value `binwidth`.



```
ggplot(data = diamonds, aes(x = price, fill = cut))+
  geom_density()
```

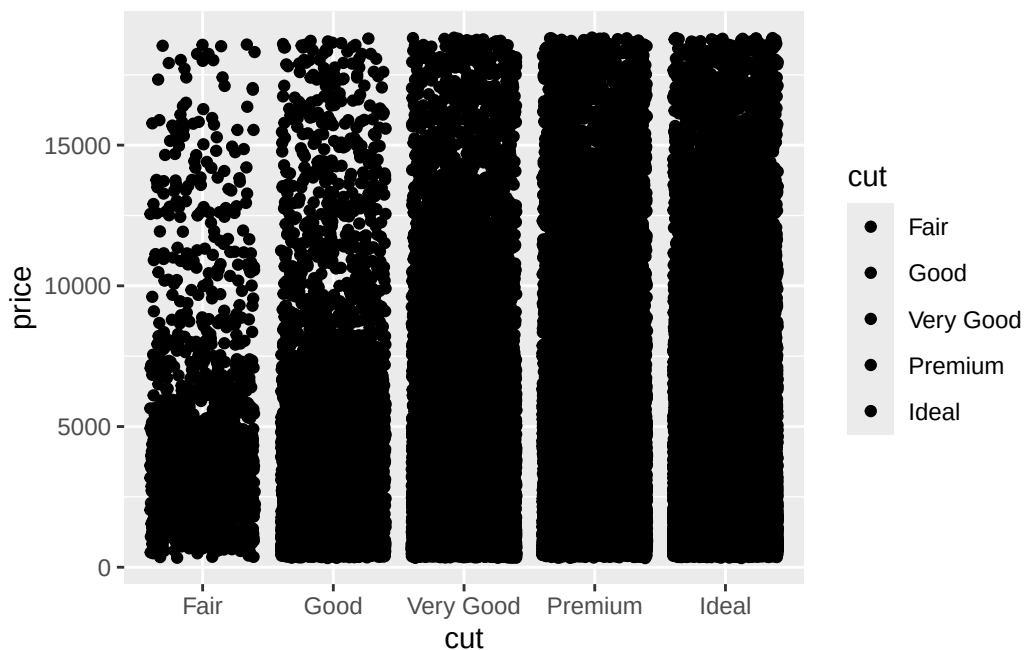


The graphs were created using the variables price and cut. In all of them, it is possible to understand the distribution of values according to the categorical variable; however, the focus of each one differs. In `geom_violin()`, the shape of each category demonstrates the distribution density. Faceted `geom_histogram()` plots are particularly useful when categories have different scales. Since they are separated along the y-axis, it is easier to observe tendencies among them. I believe the biggest difference between `geom_freqpoly()` and `geom_density()` is their final interpretation: the former uses bin counts, while the latter uses the area (density).

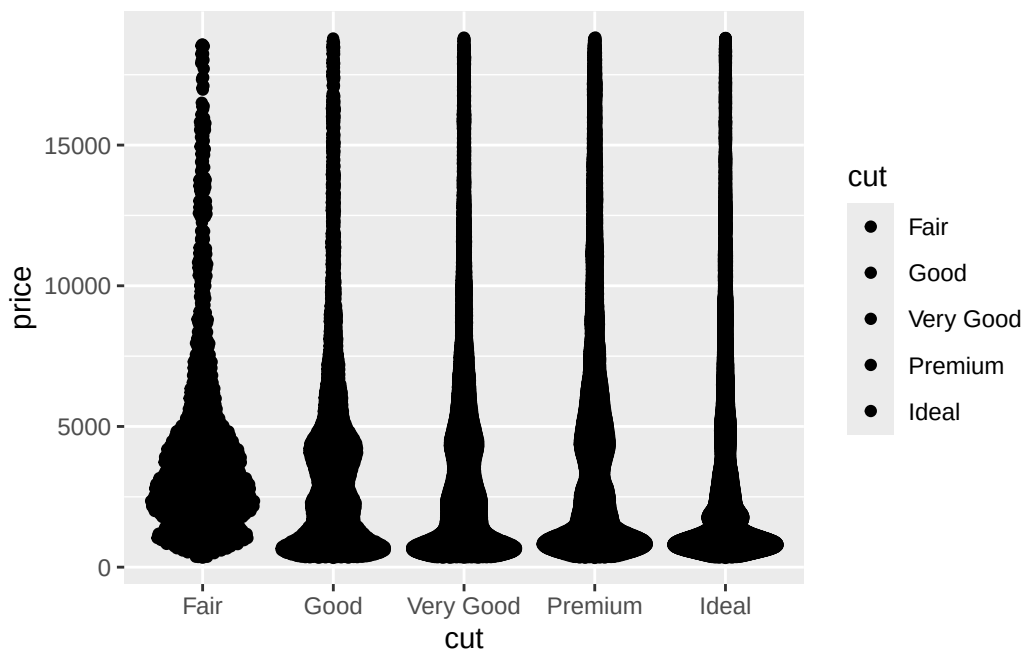
4.6 If you have a small dataset, it's sometimes useful to use `geom_jitter()` to avoid overplotting to more easily see the relationship between a continuous and categorical variable. The `ggbeeswarm` package provides a number of methods similar to `geom_jitter()`. List them and briefly describe what each one does.

```
#install.packages("ggbeeswarm")
library(ggbeeswarm)
?ggbeeswarm

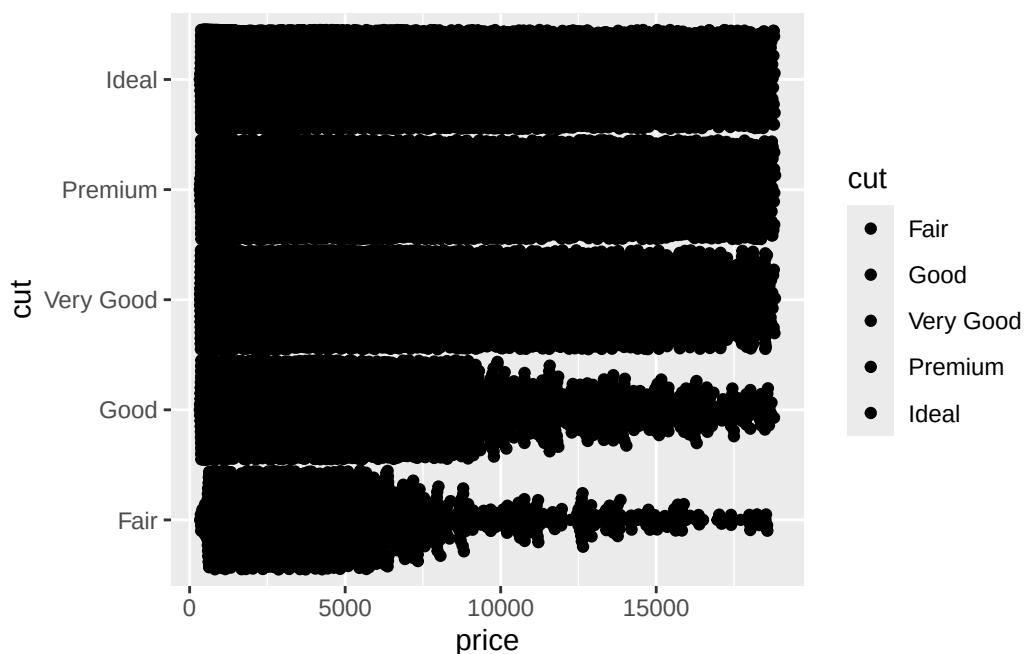
ggplot(data = diamonds, aes(x = cut, y = price, fill = cut))+
  geom_jitter()
```



```
ggplot(data = diamonds, aes(x = cut, y = price, fill = cut))+
  geom_quasirandom()
```



```
ggplot(data = diamonds, aes(x = cut, y = price, fill = cut))+
  geom_beeswarm(corral = "wrap")+
  coord_flip()
```



The section help displays this explanation for the package:

This package allows plotting of several groups of one dimensional data as a violin point/beeswarm plot in ggplot2 by arranging data points to resemble the underlying distribution.

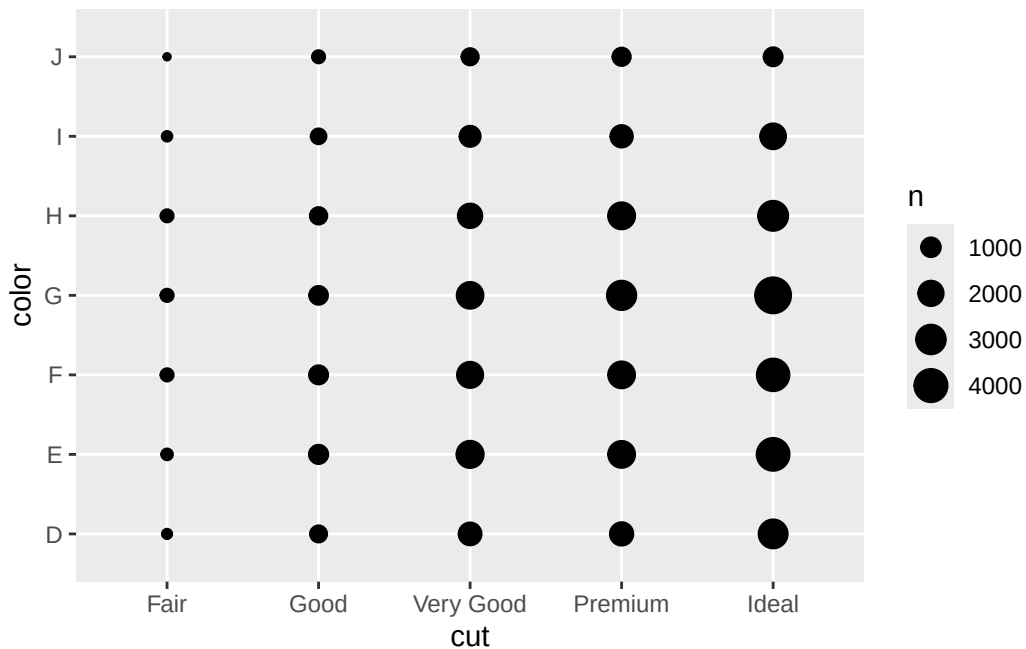
```
?geom_beeswarm
```

The ggbeeswarm package features the `geom_beeswarm()` function, which prevents points from overlapping by spreading them horizontally. It can be configured into square, hexagonal, or centered shapes. It is best suited for smaller datasets, where individual data points and their distribution remain clearly visible. It differs from `geom_jitter()` because it prevents points from overlapping, thereby improving the visualization of the data's density.

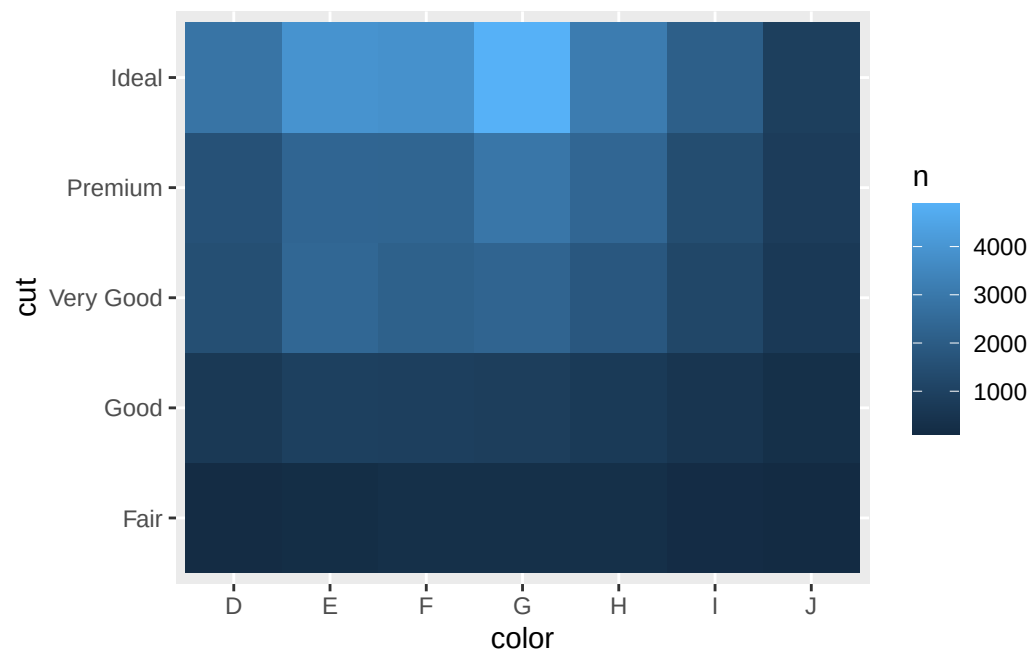
5 Exercises 10.5.2.1

5.1 How could you rescale the count dataset above to more clearly show the distribution of cut within color, or color within cut? NÃO SEI, VER DEPOIS!!

```
ggplot(diamonds, aes(x = cut, y = color)) +  
  geom_count()
```



```
diamonds |>  
  count(color, cut) |>  
  ggplot(aes(x = color, y = cut)) +  
  geom_tile(aes(fill = n))
```



```
diamonds |>
  count(color, cut) |>
  mutate(rescale_color = color/100)
```

Warning: There was 1 warning in `mutate()`.
 i In argument: `rescale\_color = color/100`.
 Caused by warning in `Ops.ordered()`:
 ! '/' is not meaningful for ordered factors

```
# A tibble: 35 x 4
  color cut          n rescale_color
<ord> <ord>    <int> <lgl>
1 D    Fair      163 NA
2 D    Good       662 NA
3 D    Very Good 1513 NA
4 D    Premium  1603 NA
5 D    Ideal    2834 NA
6 E    Fair      224 NA
7 E    Good       933 NA
8 E    Very Good 2400 NA
9 E    Premium  2337 NA
10 E   Ideal    3903 NA
# i 25 more rows
```

5.2 What different data insights do you get with a segmented bar chart if color is mapped to the x aesthetic and cut is mapped to the fill aesthetic? Calculate the counts that fall into each of the segments.

```
flights <- nycflights13::flights
```

5.3 Use `geom_tile()` together with `dplyr` to explore how average flight departure delays vary by destination and month of year. What makes the plot difficult to read? How could you improve it?

6 Exercises 10.5.3.1

6.1 Instead of summarizing the conditional distribution with a boxplot, you could use a frequency polygon. What do you need to consider when using `cut_width()` vs. `cut_number()`? How does that impact a visualization of the 2d distribution of carat and price?

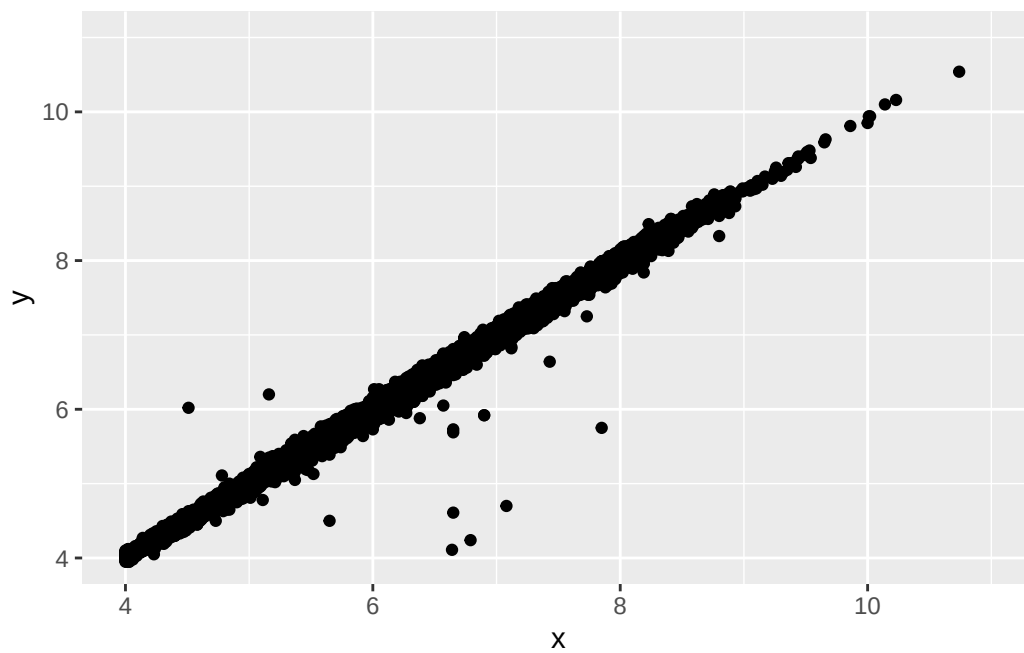
6.2 Visualize the distribution of carat, partitioned by price.

6.3 How does the price distribution of very large diamonds compare to small diamonds? Is it as you expect, or does it surprise you?

6.4 Combine two of the techniques you've learned to visualize the combined distribution of cut, carat, and price.

6.5 Two dimensional plots reveal outliers that are not visible in one dimensional plots. For example, some points in the following plot have an unusual combination of x and y values, which makes the points outliers even though their x and y values appear normal when examined separately. Why is a scatterplot a better display than a binned plot for this case?

```
diamonds |>
  filter(x >= 4) |>
  ggplot(aes(x = x, y = y)) +
  geom_point() +
  coord_cartesian(xlim = c(4, 11), ylim = c(4, 11))
```



6.6 Instead of creating boxes of equal width with `cut_width()`, we could create boxes that contain roughly equal number of points with `cut_number()`. What are the advantages and disadvantages of this approach?

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_boxplot(aes(group = cut_number(carat, 20)))
```

